Ansible

PostgreSQL. Advanced

REC

Проверить, идет ли запись

Меня хорошо видно
& слышно?

Правила вебинара

**Активно
участвуем**

**Off-topic обсуждаем
в учебной группе #postgres-advanced-2025-12**

**Задаем вопрос
в чат или голосом**

Вопросы вижу в чате,
могу ответить не сразу

Карта курса

Карта курса

-
- 1. Введение
- 2. PostgreSQL в облаках и автоматизация процессов
- 3. PostgreSQL и большие данные
- 4. Проектная работа
-

Мост от предыдущей темы

На прошлом занятии мы создали инфраструктуру с помощью **Terraform** — ВМ, сети, диски.

Но Terraform не управляет ОС и ПО внутри ВМ.

Сегодня — следующий шаг:

- **Cloud-init** — начальная настройка ОС при первом запуске
- **Ansible** — установка и конфигурация PostgreSQL
- Terraform → Cloud-init → Ansible → PostgreSQL

> Три инструмента, единый пайплайн: **Infrastructure as Code**

Проблема: масштабирование без Ansible

Зачем DBA знать Ansible?

Ручная настройка серверов — не масштабируется

- 1 сервер — SSH + руки. 10 серверов — уже больно
- Конфигурация дрейфует: каждый сервер становится «уникальным»
- Документация устаревает быстрее, чем пишется
- Откат изменений — ручной и рискованный

> **Ansible** — декларативное управление инфраструктурой: описываете «что», а не «как»

Маршрут занятия

- Cloud-init — начальная инициализация ВМ
- Terraform + Ansible — мост между IaC и конфигурацией
- Основы Ansible — инвентарь, плейбуки, роли
- Ansible для PostgreSQL — коллекция community.postgresql
- Практика — типовые сценарии DBA

Цель вебинара

К концу занятия вы сможете

Смысл

Для чего вам это уметь

Цель	Смысл
Настраивать ВМ через cloud-init и связывать с Terraform	Автоматический bootstrap серверов БД в облаке
Писать Ansible-плейбуки для установки и настройки PostgreSQL	Воспроизводимое развертывание кластеров PostgreSQL

Что такое Ansible

Ansible

Ansible — open-source платформа автоматизации конфигурации, развёртывания приложений и оркестрации. Agentless архитектура: управление через SSH без установки агентов на целевые хосты

Ansible: ключевые особенности

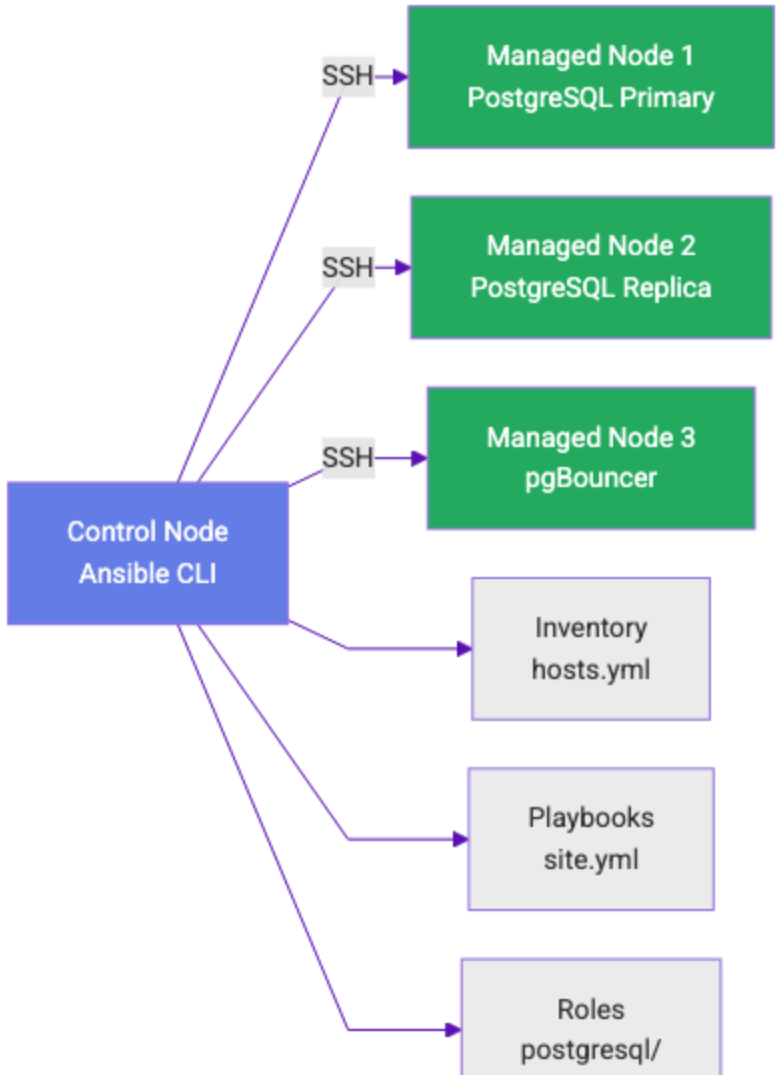
- **Agentless** — нет агентов на managed nodes, только SSH + Python
- **Декларативный** — описываете желаемое состояние, Ansible приводит к нему
- **Идемпотентный** — можно запускать многократно без побочных эффектов
- **Читаемый** — конфигурация на YAML, понятна без программирования
- **Расширяемый** — 10 000+ модулей в Ansible Galaxy

Создатель: Michael DeHaan (2012), сейчас поддерживается Red Hat

Ansible vs другие инструменты

Критерий	Ansible	Puppet/Chef
Архитектура	✓ Agentless (SSH)	Агент на каждом хосте
Язык конфигурации	✓ YAML (декларативный)	DSL (Ruby/Puppet)
Кривая обучения	✓ Низкая	Высокая
Инфраструктура	✓ Не нужен master	Требуется master-сервер
Use case	✓ Ad-hoc tasks + IaC	Долгосрочное управление конфигурацией
Push/Pull	✓ Push (контроль на control node)	Pull (агенты опрашивают master)

Ansible Core Components



Модули Ansible

Модуль Ansible

Модуль — исполняемая единица Ansible, выполняющая одну задачу на managed node: установка пакета, создание файла, управление сервисом. Модули идемпотентны и возвращают JSON с результатом

Встроенные модули: Система и пакеты

Ansible Core включает **70+ встроенных модулей**

Управление пакетами и сервисами:

- `ansible.builtin.apt` — пакеты Debian/Ubuntu
- `ansible.builtin.yum` — пакеты RHEL/CentOS
- `ansible.builtin.dnf` — пакеты Fedora/RHEL 8+
- `ansible.builtin.service` — systemd/init сервисы
- `ansible.builtin.systemd` — расширенный systemd
- `ansible.builtin.cron` — cron задачи
- `ansible.builtin.user` — управление пользователями
- `ansible.builtin.group` — управление группами

Встроенные модули: Файлы

Работа с файлами и директориями:

- `ansible.builtin.copy` — копирование файлов
- `ansible.builtin.template` — Jinja2 шаблоны
- `ansible.builtin.file` — создание файлов/каталогов, права
- `ansible.builtin.lineinfile` — редактирование строк
- `ansible.builtin.blockinfile` — вставка блоков текста
- `ansible.builtin.fetch` — копирование с удаленного хоста
- `ansible.builtin.stat` — получение информации о файле
- `ansible.builtin.find` — поиск файлов

Встроенные модули: Команды и Git

Выполнение команд:

- `ansible.builtin.command` — выполнение команд
- `ansible.builtin.shell` — выполнение через shell
- `ansible.builtin.script` — запуск локальных скриптов
- `ansible.builtin.raw` — без Python (для bootstrap)

Git и архивы:

- `ansible.builtin.git` — клонирование репозиториев
- `ansible.builtin.unarchive` — распаковка архивов
- `ansible.builtin.archive` — создание архивов

Встроенные модули: Сеть

HTTP и сетевые операции:

- `ansible.builtin.uri` — HTTP API запросы
- `ansible.builtin.get_url` — загрузка файлов по URL
- `ansible.builtin.wait_for` — ожидание порта/файла
- `ansible.builtin.ping` — проверка доступности хоста

Отладка и информация:

- `ansible.builtin.debug` — вывод переменных
- `ansible.builtin.assert` — проверка условий
- `ansible.builtin.fail` — принудительная ошибка
- `ansible.builtin.set_fact` — установка переменных

Best Practices: Модули Ansible

Tip

Всегда используйте FQCN (Fully Qualified Collection Name): `ansible.builtin.apt`, не просто `apt`

Warning

Модули `shell` и `command` не идемпотентны — используйте их только когда нет специализированного модуля

Коллекции модулей

Популярные коллекции для DBA:

Коллекция	Модули	Установка
`community.postgresql`	20+ модулей PostgreSQL	`ansible-galaxy collection install community.postgresql`
`community.mysql`	Управление MySQL/MariaDB	`ansible-galaxy collection install community.mysql`
`community.mongodb`	Управление MongoDB	`ansible-galaxy collection install community.mongodb`
`community.general`	600+ общих модулей	`ansible-galaxy collection install community.general`
`cloud.terraform`	Интеграция с Terraform	`ansible-galaxy collection install cloud.terraform`

Как работает модуль

$ansible db-primary -m ansible.builtin.apt -a "name=postgresql-17 state=present" --become

$ansible db-primary -m ansible.builtin.apt -a "name=postgresql-17 state=present" --become

Идемпотентность: первый запуск — `CHANGED`, второй — `SUCCESS` без изменений

Playbooks

Playbook

Playbook — YAML-файл с упорядоченным списком plays (задач), выполняемых на группах хостов. Playbook описывает желаемое состояние инфраструктуры декларативно

Структура Playbook

```
---
- name: Настройка PostgreSQL кластера
  hosts: postgresql
  become: true
```

Best Practices: Структура Playbook

Tip

Best Practice: Всегда используйте `name` для tasks — это улучшает читаемость логов и упрощает отладку

Warning

Используйте `become: true` только для tasks, которым нужны привилегии — не глобально для всего playbook

Запуск playbook

$ansible-playbook -i inventory.yml site.yml

$ansible-playbook site.yml --check

$ansible-playbook site.yml --tags postgresql --limit db-primary

Опции: `--check` (dry-run), `--tags` (выборочный запуск), `--limit` (фильтр хостов)

Roles: переиспользуемые компоненты

Role — структурированная коллекция tasks, handlers, templates, files для переиспользования

```
roles/postgresql/
├── tasks/
│   └── main.yml
├── handlers/
│   └── main.yml
├── templates/
│   ├── postgresql.conf.j2
│   └── pg_hba.conf.j2
├── files/
│   └── init.sql
```

Использование ролей

```
---
- name: Развернуть PostgreSQL HA кластер
  hosts: postgresql
```

```
---
- name: Развернуть PostgreSQL HA кластер
  hosts: postgresql
  roles:
    - role: geerlingguy.postgresql
      vars:
        postgresql_version: "17"
```

Ansible Tower / AWX

Ansible Tower

Ansible Tower — enterprise web UI и API для Ansible (платный). **AWX** — open-source upstream версия. Централизованное управление playbooks, инвентарями, RBAC, аудит и планировщик

Возможности Ansible Tower / AWX

- **Web UI** — запуск playbooks из браузера без SSH к control node
- **RBAC** — ролевая модель доступа (кто может запускать что и где)
- **Credentials Vault** — безопасное хранение SSH-ключей, паролей, токенов облаков
- **Job Scheduling** — cron-like планировщик для регулярных задач (backup, patching)
- **REST API** — интеграция с CI/CD (Jenkins, GitLab CI, GitHub Actions)
- **Audit & Logging** — полная история запусков playbooks, изменений, кто что запустил
- **Surveys** — интерактивные формы для параметров playbook (версия БД, размер диска)

Ansible Tower vs AWX

Критерий	Ansible Tower	AWX
Поддержка	✓ Enterprise support от Red Hat	Community support
Цена	Платная подписка (от $5000/год)	✓ Бесплатно
Релизы	✓ Стабильные (раз в квартал)	Bleeding edge (каждые 2-3 недели)
GUI	✓ Полированный UI	Функциональный UI
Upgrade path	✓ Официальный путь обновления	Может сломаться между версиями
Use case	✓ Production enterprise	Dev/Test или малый бизнес

Когда использовать Tower/AWX

✅ **Подходит:**

- Команда > 5 DBA: нужен RBAC и аудит
- Регулярные задачи: ежедневные backup, патчинг ОС
- Compliance: требования к логированию изменений
- CI/CD интеграция: автодеплой после git push

❌ **Избыточно:**

- Небольшая команда (1-2 DBA) — ansible CLI достаточно
- Редкие задачи — запуск вручную проще
- Прототипирование — Tower добавляет сложность

> **Tip:** Начните с ansible CLI, переходите на Tower при росте команды или требований compliance.

Ansible Galaxy

Ansible Galaxy

Ansible Galaxy — публичный репозиторий community-разработанных ролей и коллекций. Hub для обмена автоматизацией: скачиваете готовые роли вместо написания с нуля

Поиск и установка с Galaxy

```
$ansible-galaxy search postgresql
$ansible-galaxy role install geerlingguy.postgresql
$ansible-galaxy collection install community.postgresql
```

requirements.yml: зависимости проекта

Вместо ручной установки — декларативный файл зависимостей:

```yaml
---
roles:
  - name: geerlingguy.postgresql
    version: "4.0.3"
  - src: https://github.com/myorg/custom-postgres-role
    version: main
```

```yaml
---
roles:
```

Популярность Galaxy

36,934	*3,943*	*20,000+*
Roles	Collections	Community Authors

Top PostgreSQL роли на Galaxy

Role	⭐ Stars	Downloads	Use Case
`geerlingguy.postgresql`	850+	2M+	Production-ready, простая установка
`anxs.postgresql`	500+	500K+	Advanced replication, streaming
`robertdebock.postgres`	200+	100K+	Molecule-tested, CI/CD ready
`vitabaks.autobase`	150+	50K+	HA clusters (Patroni + etcd)

Рекомендация:

- **Начинающие:** `geerlingguy.postgresql` — простая, хорошо документирована
- **HA кластеры:** `vitabaks.autobase` — из коробки Patroni
- **Custom:** Напишите свою роль, используя модули `community.postgresql`

Terraform + Ansible: мост между мирами

terraform-provider-ansible

terraform-provider-ansible — провайдер Terraform от Red Hat, позволяющий описывать Ansible-инвентарь и запускать плейбуки прямо из Terraform (v1.3.0)

Разделение ответственности

- **Terraform** — создание ресурсов (ВМ, сети, диски)
- **Cloud-init** — начальная настройка ОС при первом запуске
- **Ansible** — конфигурация ПО, управление жизненным циклом

День 0 vs День 1+: разделение ответственности

День 0: Инфраструктура (Terraform + Cloud-init)

- Создание ВМ, сетей, дисков
- Начальная настройка ОС (пакеты, пользователи, SSH)
- Выполняется **один раз** при создании ресурсов

День 1+: Конфигурация (Ansible)

- Установка и настройка приложений
- Управление конфигурацией и состоянием
- Выполняется **многократно** при обновлениях

Note

Cloud-init готовит ВМ для работы с Ansible — это мостик между Terraform и Ansible

Cloud-init: День 0 — подготовка ВМ

Cloud-init

Cloud-init — индустриальный стандарт инициализации виртуальных машин при первом запуске. Поддерживается всеми основными облаками: AWS, GCP, Azure, Yandex Cloud

Как работает cloud-init

Стадия	Что происходит
1. Detect	Определение облачной платформы (ds-identify)
2. Local	Настройка сети (DHCP fallback)
3. Network	Обработка user-data: диски, SSH-ключи, пользователи
4. Config	Выполнение runcmd, настройка NTP, timezone
5. Final	Установка пакетов, Ansible, скрипты пользователя

Warning

Cloud-init работает **только при первом запуске ВМ** — для повторной настройки нужно пересоздать ВМ или очистить `/var/lib/cloud`

Cloud-config: формат конфигурации

Файл **обязательно** начинается с `#cloud-config`:

```
#cloud-config
users:
  - name: deploy
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - ssh-rsa AAAA...
```

Cloud-init: bootstrap PostgreSQL

```
#cloud-config
package_update: true
```

Cloud-init + Ansible: нативная интеграция

Начиная с cloud-init **22.3** — встроенный модуль `cc_ansible`:

```
#cloud-config
ansible:
```

```
#cloud-config
ansible:
  package_name: ansible-core
  install_method: pip
```

Cloud-init в Yandex Cloud

Yandex Cloud поддерживает cloud-init через **metadata service**:

```
resource "yandex_compute_instance" "db" {
  name = "pg-server"
  # ...
```

```
resource "yandex_compute_instance" "db" {
  name = "pg-server"
  # ...
  metadata = {
    ssh-keys  = "ubuntu:${file("~/.ssh/id_rsa.pub")}"
    user-data = file("cloud-init.yaml")
```

Cloud-init: когда использовать

✅ Подходит

- Начальная настройка ОС
- SSH-ключи, пользователи
- Установка пакетов
- Bootstrap Ansible

❌ Не подходит

- Текущее управление конфигурацией
- Развертывание приложений
- Оркестрация кластеров
- Изменения после первого запуска

Что важно запомнить: cloud-init

Cloud-init — для первого запуска

- Работает один раз при создании ВМ
- Формат `#cloud-config` (YAML)

Нативная поддержка Ansible

- Модуль `cc_ansible` с версии 22.3
- `ansible-pull` для локального запуска

Интеграция с облаками

- `metadata.user-data` в Yandex Cloud
- Поддержка AWS, GCP, Azure, OpenStack

Вопросы?

+ если есть вопросы	− если вопросов нет

Подключение провайдера Ansible

После того как Cloud-init подготовил ВМ (Python, SSH-ключи), можно подключить Ansible через Terraform:

```
terraform {
  required_providers {
    ansible = {
      source  = "ansible/ansible"
      version = "~> 1.3.0"
    }
  }
}
```

Ресурсы провайдера:

- `ansible_host` — хост инвентаря
- `ansible_group` — группа с переменными

Провайдер Ansible: предостережение

Warning

Запуск `ansible_playbook` из Terraform может блокировать `terraform apply` на время выполнения плейбука — для больших инфраструктур лучше запускать Ansible отдельно

Рекомендации:

- Используйте `ansible_host` и `ansible_group` для динамического инвентаря
- Запускайте плейбуки вручную или через CI/CD после `terraform apply`
- Для простых случаев `ansible_playbook` подходит, но учитывайте время выполнения

ansible_host и ansible_group

```
resource "ansible_host" "db_server" {
  name   = yandex_compute_instance.db.network_interface.0.nat_ip_address
  groups = ["postgresql", "production"]
  variables = {
    ansible_user         = "ubuntu"
    ansible_ssh_private_key_file = "~/.ssh/id_rsa"
    postgresql_version   = "17"
  }
}

resource "ansible_group" "db_cluster" {
  name     = "db_cluster"
  children = ["pg_primary", "pg_replicas"]
  variables = {
    ansible_become = "true"
    db_port        = "5432"
  }
}
```

Динамический инвентарь из Terraform

Коллекция `cloud.terraform` читает state Terraform:

```
# inventory.yml
plugin: cloud.terraform.terraform_provider
project_path: /path/to/terraform/project
```

Установка:

```
ansible-galaxy collection install cloud.terraform
```

> Инвентарь **автоматически** синхронизируется с Terraform state — добавили ВМ, она сразу видна в Ansible

Динамический инвентарь: пример использования

```
$ansible-galaxy collection install cloud.terraform
$ansible-playbook -i inventory.yml site.yml
```

Способы передачи инвентаря

Критерий	terraform-provider-ansible	templatefile() + local_file
Тип инвентаря	✓ Динамический (автосинхронизация)	Статический файл
Ресурсы Terraform	✓ `ansible_host` / `ansible_group`	`local_file` с templatefile()
Обновление при apply	✓ Автоматическое	Требует пересоздания файла
Сложность настройки	Средняя (провайдер v1.3.0+)	✓ Низкая (стандартный Terraform)
Use case	✓ Большие проекты, частые изменения	Небольшие статичные проекты

Генерация инвентаря: Terraform

```
resource "local_file" "inventory" {
  content = templatefile("inventory.tftpl", {
    primary_ip  = yandex_compute_instance.primary
                    .network_interface.0.nat_ip_address
    replica_ips = yandex_compute_instance.replica[*]
                    .network_interface.0.nat_ip_address
  })
  filename = "${path.module}/inventory.ini"
}
```

Генерация инвентаря: шаблон

Шаблон `inventory.tftpl`:

```
[pg_primary]
${primary_ip} ansible_user=ubuntu

[pg_replicas]
%{ for ip in replica_ips ~}
${ip} ansible_user=ubuntu
%{ endfor ~}
```

> `templatefile()` подставляет значения Terraform-переменных в шаблон при `terraform apply`

Что важно запомнить: Terraform + Ansible

terraform-provider-ansible v1.3.0

- Ресурсы `ansible_host`, `ansible_group`, `ansible_playbook`
- Коллекция `cloud.terraform` для динамического инвентаря

Разделение ответственности

- Terraform: Day 0 (инфраструктура)
- Ansible: Day 1+ (конфигурация)

Альтернативный подход

- `templatefile()` + `local_file` для статического инвентаря

Вопросы?

+ если есть вопросы	− если вопросов нет

Основы Ansible для DBA

Ansible

Ansible — система управления конфигурацией без агентов (agentless). Подключается по SSH, выполняет задачи декларативно и идемпотентно. Текущая версия: ansible-core 2.20

Ключевые компоненты Ansible

Control Node — машина, с которой запускается Ansible

Managed Nodes — целевые серверы, управляемые по SSH

Inventory — список хостов и групп с переменными

Playbook — YAML-файл с описанием желаемого состояния

Форматы инвентаря

INI-формат

```
[pg_primary]
db-primary ansible_host=10.0.1.10

[pg_replicas]
db-replica1 ansible_host=10.0.1.11

[postgresql:children]
pg_primary
pg_replicas
```

YAML-формат

```
all:
  children:
```

Первый плейбук

```
---
- name: Установка PostgreSQL
  hosts: postgresql
  become: true
```

FQCN: полное имя модуля

С Ansible 2.10+ модули объединены в **коллекции**:

```
namespace.collection.module
```

Старый синтаксис	FQCN
`apt`	`ansible.builtin.apt`
`service`	`ansible.builtin.service`
`template`	`ansible.builtin.template`
`postgresql_db`	`community.postgresql.postgresql_db`

> **Warning:** ansible-lint требует FQCN — старый синтаксис deprecated

Структура каталогов проекта

```
ansible-postgresql/
├── ansible.cfg
├── inventory/
│   ├── hosts.yml
│   ├── group_vars/
│   │   ├── all.yml
│   │   └── postgresql.yml
│   └── host_vars/
│       └── db-primary.yml
├── playbooks/
│   └── site.yml
├── roles/
│   └── postgresql/
```

Handlers: перезапуск по требованию

```
tasks:
```

Шаблоны Jinja2

Файл `templates/postgresql.conf.j2`:

```
# Ansible managed - do not edit manually
listen_addresses = '{{ pg_listen_addresses | default("localhost") }}'
port = {{ pg_port | default(5432) }}
max_connections = {{ pg_max_connections | default(100) }}
shared_buffers = {{ pg_shared_buffers | default("128MB") }}
effective_cache_size = {{ pg_effective_cache_size | default("4GB") }}
work_mem = {{ pg_work_mem | default("4MB") }}
wal_level = {{ pg_wal_level | default("replica") }}
```

Переменные задаются в `group_vars/postgresql.yml`:

```yaml
pg_listen_addresses: "*"
pg_max_connections: 200
pg_shared_buffers: "2GB"
```

Ansible Vault: шифрование секретов

Команды шифрования:

```
# Зашифровать строку
ansible-vault encrypt_string 'S3cretP@ss' --name 'pg_password'

# Зашифровать файл
ansible-vault encrypt group_vars/postgresql/vault.yml

# Запустить плейбук с расшифровкой
ansible-playbook site.yml --ask-vault-pass
```

Ansible Vault: использование в плейбуке

Подключение зашифрованных переменных:

```yaml
vars_files:
  - group_vars/postgresql/vault.yml

tasks:
  - community.postgresql.postgresql_user:
      name: app_user
      password: "{{ pg_password }}"
```

Ansible Vault: пример работы

```
$ansible-vault encrypt_string 'S3cretP@ss' --name 'pg_password'
$ansible-vault encrypt group_vars/postgresql/vault.yml
$ansible-playbook site.yml --ask-vault-pass
```

Что важно запомнить: основы Ansible

Agentless архитектура

- Подключение по SSH, Python на managed nodes
- ansible-core 2.20.2, Python 3.11+ на control node

Ключевые концепции

- Inventory (INI/YAML), Playbooks, Roles, Handlers
- FQCN обязателен: `community.postgresql.postgresql_db`

Безопасность

- Ansible Vault для шифрования паролей БД

Вопросы?

+ если есть вопросы	− если вопросов нет

community.postgresql: управление БД как кодом

Коллекция community.postgresql

Установка:

```
ansible-galaxy collection install community.postgresql
```

Или через `requirements.yml`:

```yaml
collections:
  - name: community.postgresql
    version: ">=4.0.0"
```

Требования на managed node:

- PostgreSQL установлен
- `psycopg2 >= 2.5.1` или `psycopg3 >= 3.1.8`

```
apt install python3-psycopg2
```

Основные модули коллекции

Модуль	Назначение
`postgresql_db`	Создание/удаление баз данных
`postgresql_user`	Управление ролями и паролями
`postgresql_privs`	GRANT/REVOKE привилегий
`postgresql_query`	Выполнение SQL-запросов
`postgresql_pg_hba`	Управление pg_hba.conf
`postgresql_alter_system`	ALTER SYSTEM SET (PG 14+)
`postgresql_info`	Сбор информации о сервере
`postgresql_ext`	Управление расширениями
`postgresql_ping`	Проверка доступности сервера

Дополнительные модули

Модуль	Назначение
`postgresql_slot`	Управление слотами репликации
`postgresql_publication`	Публикации логической репликации
`postgresql_subscription`	Подписки логической репликации
`postgresql_schema`	Управление схемами
`postgresql_tablespace`	Управление табличными пространствами
`postgresql_idx`	Создание/удаление индексов
`postgresql_copy`	COPY между файлом и таблицей
`postgresql_script`	Запуск SQL-скриптов

Создание БД и пользователей

```
- name: Создать базу данных
  community.postgresql.postgresql_db:
    name: myapp_production
    encoding: UTF-8
    lc_collate: en_US.UTF-8
    template: template0
    state: present
  become_user: postgres

- name: Создать пользователя
```

Управление pg_hba.conf

```
- name: Разрешить подключение приложения
  community.postgresql.postgresql_pg_hba:
    dest: /etc/postgresql/17/main/pg_hba.conf
    contype: host
    databases: myapp_production
    users: myapp_user
    address: 10.0.2.0/24
    method: scram-sha-256
    state: present
  notify: Reload PostgreSQL
```

postgresql_alter_system (PG 14+)

```
- name: Настройка shared_buffers
  community.postgresql.postgresql_alter_system:
    name: shared_buffers
    value: "{{ (ansible_memtotal_mb * 0.25) | int }}MB"
  become_user: postgres
  notify: Restart PostgreSQL
```

Сбор информации и запросы

```
- name: Информация о сервере
  community.postgresql.postgresql_info:
    filter:
      - version
      - databases
      - settings
  become_user: postgres
  register: pg_info
```

Автоматизация бэкапов

```
- name: Создать каталог для бэкапов
  ansible.builtin.file:
    path: /var/backups/postgresql
    state: directory
    owner: postgres
    mode: "0700"
```

```
- name: Создать каталог для бэкапов
  ansible.builtin.file:
```

Автоматизация бэкапов: расписание

```yaml
- name: Ежедневный бэкап по cron
  ansible.builtin.cron:
    name: "Daily PostgreSQL backup"
    minute: "0"
    hour: "2"
    job: >
      pg_dump -Fc myapp_production |
      gzip > /var/backups/postgresql/
      myapp_$(date +\%Y-\%m-\%d).sql.gz
    user: postgres
```

> **Tip:** Для production рассмотрите **pgBackRest** или **WAL-G** вместо pg_dump — они поддерживают инкрементальные бэкапы

Логическая репликация

```
# На primary: создать публикацию
- name: Создать публикацию
  community.postgresql.postgresql_publication:
    db: myapp_production
    name: myapp_pub
    alltables: true
    state: present
  become_user: postgres
```

Что важно запомнить: community.postgresql

Коллекция v4.2.0

- 20+ модулей для полного управления PostgreSQL
- Требует psycopg2/psycopg3 на managed node

Ключевые модули

- `postgresql_db`, `postgresql_user`, `postgresql_pg_hba`
- `postgresql_alter_system` (замена `postgresql_set` для PG 14+)

Важные паттерны

- `become_user: postgres` для peer-аутентификации
- Ansible Vault для паролей

Вопросы?

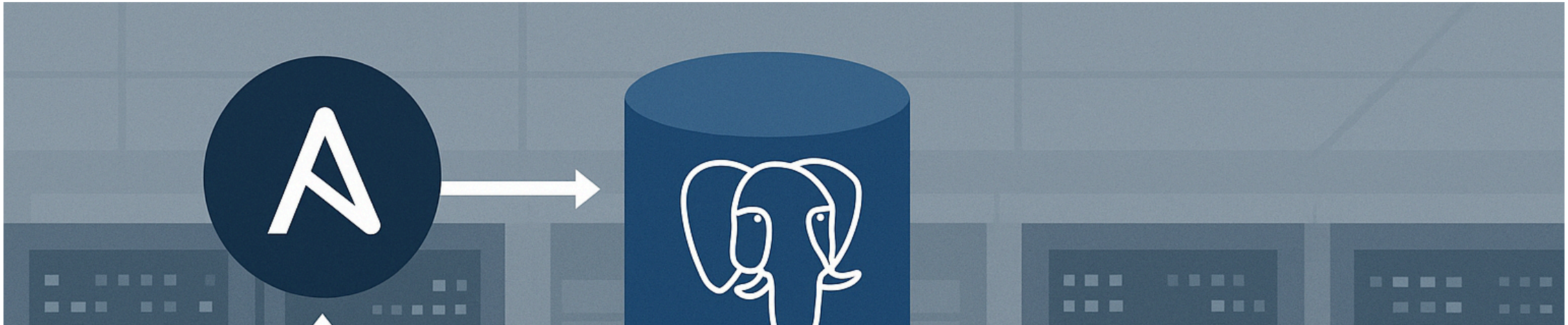
+ если есть вопросы	− если вопросов нет

Типичные ошибки и best practices

Production-ready Ansible

Практика — это не просто код, это **культура**:

- Версионируйте всё: Ansible playbooks, roles, requirements
- Тестируйте: molecule, ansible-lint перед commit
- Документируйте: README, примеры использования
- Мониторьте: собирайте логи, метрики, оповещения
- Итерируйте: регулярные обновления, security patches



become_user для PostgreSQL

✅ Правильно

```yaml
- postgresql_db:
    name: mydb
  become: true
  become_user: postgres
```

Или на уровне play:

```yaml
- hosts: postgresql
  become_user: postgres
```

❌ Неправильно

```yaml
- postgresql_db:
    name: mydb
```

Идемпотентность SQL-запросов

✅ Используйте модули

```
# Идемпотентно!
- postgresql_ext:
    name: pg_stat_statements
    db: mydb
    state: present
```

Модули сами проверяют текущее состояние

❌ Сырой SQL

```
# changed ВСЕГДА = true
- postgresql_query:
    query: >
      CREATE EXTENSION
```

Подключение: unix socket vs TCP

✅ Unix socket (peer auth)

```
# login_host НЕ указан
- postgresql_db:
    name: mydb
  become_user: postgres
```

Используется peer-аутентификация

❌ localhost = TCP

```
# TCP: нужен пароль!
- postgresql_db:
    name: mydb
    login_host: localhost
    login_user: postgres
```

Готовые роли для PostgreSQL

geerlingguy.postgresql v4.0.2 — установка и базовая настройка PostgreSQL, управление БД и пользователями

vitabaks.autobase v2.5.2 — полноценный HA-кластер: Patroni + etcd + HAProxy + PgBouncer

Собственные роли — максимальная гибкость, используйте модули community.postgresql как строительные блоки

Полный пайплайн: от ВМ до PostgreSQL

1. **Инфраструктура** — `terraform apply`
2. **Коллекции** — `ansible-galaxy collection install -r requirements.yml`
3. **Конфигурация** — `ansible-playbook -i inventory.yml site.yml`
4. **Проверка** — `ansible postgresql -m postgresql_ping`

> Весь пайплайн можно запустить из CI/CD (GitLab CI, GitHub Actions)

Что важно запомнить: best practices

Аутентификация

- `become_user: postgres` + unix socket для peer auth
- `login_host: localhost` = TCP (нужен пароль!)

Идемпотентность

- Специализированные модули вместо `postgresql_query`
- `postgresql_query` — only когда нет подходящего модуля

Готовые решения

- autobase для HA-кластеров, geerlingguy для простых случаев

Вопросы?

+ если есть вопросы	− если вопросов нет

Полезные материалы

Документация:

- Ansible Documentation — официальная документация
- community.postgresql — документация коллекции
- cloud-init Documentation — документация cloud-init

Инструменты:

- terraform-provider-ansible — провайдер Terraform
- vitabaks/autobase — PostgreSQL HA с Ansible
- geerlingguy.postgresql — роль для PostgreSQL

Результаты

Что узнали

Применение

Как использовать на практике

Результат	Применение
Настраивать ВМ через cloud-init и Terraform	Автоматический bootstrap PostgreSQL-серверов
Писать Ansible-плейбуки для PostgreSQL	Воспроизводимые развертывания в production

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу ✏️❤️

Приглашаем на следующий вебинар: 20.02.2025 "Управление PostgreSQL в Yandex Cloud"

Ссылка на вебинар будет в ЛК за 15 минут

Материалы к занятию в ЛК — можно изучать

Обязательный материал обозначен красной лентой